

Socket Event: CREATE_COMMENT

Functional Requirements Covered

- `_FR10_` - Logged-in users can comment on posts
- `_FR11_` - Each comment includes: author, text and date
- `_FR12_` - Users can edit or delete their own comments

Event Purpose

The `CREATE_COMMENT` socket event allows an authenticated user to add a comment to an existing post.

Authentication

This event requires authentication (a valid JWT). The `authorId` is inferred from the user data in the token:

``this.user.sub``

Payload Example

```
```js
{
 postId: "683bf8c132067f2f5dbeea3c",
 text: "This is a test comment",
 authorId: "683bf83781b5203edd055b0f"
}
```
```

Input Validation

Handled by: `_CreateCommentValidator_`

- `postId`: required 24-character hex string
- `text`: required non-empty string
- `authorId`: required 24-character hex string

Invalid input throws a `BadRequestException` with detailed message.

Success Server Response

```
```js
{
 success: true,
 data: {
 _id: '6842021cfc9e0dab7ed97957',
 text: 'Test comment',
 authorId: '6842021cfc9e0dab7ed9794b',
 authorUsername: 'user1749879819768',
 postId: '6842021cfc9e0dab7ed97953',
 created_at: '2025-06-05T20:46:20.802Z'
 }
}
```
```

Error Server Response

```
```js
{
 success: false,
 error: {
 code: 400,
 status: 400,
 message: "\"text\" is required"
 }
}
```
```

Flow

Handled by: `_CreateCommentEventHandler_`

The `CreateCommentEventHandler` handles the ``CREATE_COMMENT socket event``. It checks that the user is logged in, validates the data sent by the client and creates a new comment using the service. Before sending a confirmation back to the user, it enriches the comment with the author's username using the ``addAuthorToComment`` function. Then, it sends the enriched comment back to the user and notifies other users connected to the post about the new comment in real time.

In ``CommentService.createComment``

- Checks if the post exists using ``postRepository.findById``
If not found, throws a `NotFoundException`
- Builds a list of relevant user IDs by combining `post.userIds` with `authorId` (if not already included)
- Filters only active users from that list by querying `userSchema` with status: ``userStatus.ACTIVE``
- Sets `userIds` in the comment to the list of active users
- Creates the comment using ``commentRepository.create(data)``
- Increments the post's `total_comments` using ``postRepository.incrementCommentCount``
- Returns the created comment

In ``CommentRepository.create``

- Creates the comment in the database
- Throws a custom error if saving fails
- Returns the created comment

Broadcasted Events

After a comment is successfully created:

``COMMENT_CREATED``: Sent only to the author of the comment

``COMMENT_SHARED``: Sent to all other active users (from `userIds`) associated with the post, excluding the author

```
```js
```

```
this.socket.to(userId).emit(eventName, comment.toJSON());
```
```

Only users with ``userStatus.ACTIVE`` (filtered earlier in `CommentService`) are included in `userIds`. As a result, only active users receive the broadcasted socket events

Socket Event: UPDATE_COMMENT

Functional Requirements Covered

- `_FR12_` - Users can edit their own comments

Event Purpose

The ``UPDATE_COMMENT` socket event` allows an authenticated user to update the content of a comment.

Authentication

This event requires authentication (a valid JWT). The `authorId` is inferred from the user data in the token:

``this.user.sub``

Payload Example

```
`` `js
{
  commentId: "683bf83781b5203edd055b1b",
  text: "Updated comment content"
}
`` `
```

Input Validation

Handled by: `_UpdateCommentValidator_`

- `commentId`: required, 24-character hexadecimal string
- `text`: required, non-empty trimmed string

Invalid input will throw a `BadRequestException` with a descriptive message and error code.

Success Server Response

```
```js
{
 success: true,
 data: {
 _id: '684298032fc33fc6cb94009c',
 text: 'Updated comment',
 authorId: '684298032fc33fc6cb940091',
 authorUsername: 'user1749879819768',
 postId: '684298032fc33fc6cb94009a',
 created_at: '2025-06-06T07:25:55.870Z'
 }
}
```
```

Error Server Response

```
```js
{
 success: false,
 error: {
 code: 400,
 status: 400,
 message: "\"text\" is not allowed to be empty"
 }
}
```
```

Flow

Handled by: `_UpdateCommentEventHandler_`

The `UpdateCommentEventHandler` listens for the ``UPDATE_COMMENT socket event``. It first checks if the user is authenticated, then validates the provided `commentId` and `text`. If the data is valid, it updates the comment using ``commentService.updateComment``, enriches it with the author's username via the ``addAuthorToComment`` function, and sends the normalized result back to the user via the socket acknowledgment. If there's an error (invalid input or unauthorized access), a clear error response is returned.

In ``CommentService.updateComment``:

- Retrieves the comment by ID using ``commentRepository.findById``
- Throws a `NotFoundException` if the comment does not exist
- Updates the comment's text with the new content
- Calls `commentRepository.update` to save the changes, ensuring the comment belongs to the authenticated user
- Returns the updated comment

In ``CommentRepository.update``:

- Finds the comment by `\_id` and ensures the `authorId` matches the authenticated user
- Updates the text field and returns the updated comment
- Throws:
 - `NotFoundException` if the comment does not exist or doesn't belong to the user
 - `MongoInternalException` for internal DB errors

Broadcasted Events

Unlike `CREATE_COMMENT`, the ``UPDATE_COMMENT event`` does not emit any broadcast events to other users. The server returns the updated comment only to the user who performed the update, using the ack callback.

Socket Event: DELETE_COMMENT

Functional Requirements Covered

- `_FR12_` - Users can delete their own comments

Event Purpose

The ``DELETE_COMMENT` socket event` allows an authenticated user to delete their own comment. Upon successful deletion:

- The comment's status is updated to "deleted" (soft delete)
- The corresponding post's `total_comments` counter is decremented
- An updated post notification (`POST_UPDATED`) is broadcast to all users associated with the post

Authentication

This event requires authentication (a valid JWT). The `authorId` is inferred from the user data in the token:
``this.user.sub``

Payload Example

```
```js
{
 commentId: '683df1e3663dac33d2e0033a';
}
```
```

Input Validation

Validated by: `_DeleteCommentValidator_`

- `commentId`: 24-character hexadecimal string, required

Invalid input results in a `BadRequestException` with a clear message and error code

Success Server Response

```
```js
{
 deleted: true,
 comment: {
 _id: '6842c9916c5b08cfab6933e3',
 postId: '6842c9916c5b08cfab6933e1',
 text: 'Text comment',
 }
}
```

```
 authorId: '6842c9916c5b08cfab6933d8',
 userIds: [],
 authorUsername: 'user1749879819768',
 status: 'deleted'
 userIds: ['6842c9916c5b08cfab6933d8'],
 created_at: '2025-06-06T10:57:21.782Z',
 }
}
```

## ## Error Server Response

Input validation failure example:

```
```js
{
  success: false,
  error: {
    code: 400,
    status: 400,
    message: "\"commentId\" length must be 24 characters long"
  }
}
```

or Comment not found or not deleted:

```
```js
{
 success: false,
 deleted: false
}
```

## ## Flow

Handled by: `_DeleteCommentEventHandler_`

The DeleteCommentEventHandler listens for the ``DELETE_COMMENT` socket event``.

It authenticates the user, validates the comment ID, and calls the ``commentService.deleteComment`` method to perform a soft delete of the comment. If the deletion fails, it acknowledges



the client with { success: false, deleted: false }. If successful, it returns the deleted comment data. Retrieves the updated post and broadcasts a POST\_UPDATED event to all users associated with that post, notifying them of the updated comment count.

In ``CommentRepository.delete``

- Performs a soft delete by setting the comment's status to "deleted" only if the comment belongs to the authenticated user
- Returns the updated comment or null (if not found or not deletable)

In ``CommentService.deleteComment``

- Calls ``CommentRepository.delete`` to soft-delete the comment if owned by the user
- If deletion fails or comment not found, returns { success: false, deleted: false }
- If deletion succeeds and the comment has an associated postId, decrements the post's total\_comments count
- Returns the deleted comment

## ## Broadcasted Events

- After a successful deletion, a ``POST_UPDATED event`` is emitted only to the user who deleted the comment.
- No other users are notified
- This event includes the updated post data, including the new total\_comments value.